ARQUITETURA DE SOFTWARE

Guia Completo de Revisão e Síntese Teórica (Aulas 1 a 5)

Prof. Dr. Pedro Henrique Nogueira Pizzutti

Objetivo do Material: *Este documento consolida todos os tópicos, definições clássicas, analogias, atributos de qualidade, anti-padrões e estudos de caso apresentados nas Aulas 1 a 5. Foi estruturado para oferecer uma revisão técnica densa e rápida para a prova.*

MÓDULO 1 • INTRODUÇÃO À ARQUITETURA DE SOFTWARE

1.1 Definição Clássica (Bass, Clements e Kazman, 2003)

A arquitetura de software é o **conjunto de estruturas necessárias para raciocinar sobre o sistema**, reunindo seus **elementos**, as **relações** entre eles e as **propriedades** de ambos (BASS; CLEMENTS; KAZMAN, 2003, p. 4).

- **Elementos:** As partes do sistema (módulos de código, componentes em tempo de execução, microsserviços).
- **Relações:** Como esses elementos se conectam, comunicam e trocam dados entre si.
- **Propriedades:** As características de funcionamento, comportamento e responsabilidades de cada elemento.

1.2 O que NÃO é Arquitetura de Software Isoladamente

- **Não é só escolher tecnologia:** Optar por React, Java ou Spring não resolve como o sistema é particionado nem como as partes dialogam.
- **Não é só organizar pastas:** Pastas bem dispostas revelam organização física, mas não capturam as decisões sistêmicas.
- **Não é só infraestrutura/deploy:** Rodar em contêineres Docker ou na AWS resolve hospedagem, mas não define a lógica das relações.

1.3 Estrutura vs Decisão e o Custo no Ciclo de Vida

A **Estrutura** é o resultado visível e observável hoje no código (ex: diagrama de módulos). A **Decisão** é o raciocínio, trade-offs e escolhas que deram origem a essa estrutura. A maior fatia do custo de um software concentra-se na **manutenção e evolução** em produção, e não no desenvolvimento inicial.

1.4 Categorias de Mudança no Software

Categoria	O que Afeta	Exemplo Prático
Local	Um único elemento do sistema (mais barata e segura).	Alterar regra de desconto dentro de um módulo existente.
Não Local	Vários elementos, mantendo a mesma abordagem arquitetural.	Nova regra + campo no banco + ajuste na tela de resultado.
Arquitetural	A forma como os elementos interagem entre si (mais cara).	Migrar de cliente-servidor para modelo ponto a ponto (P2P).

1.5 Casos Reais de Impacto e Trade-offs

- **Knight Capital (2012):** Função desativada há quase 10 anos foi reaproveitada; um servidor rodou código desatualizado e a empresa perdeu US$ 460 milhões em 45 minutos. *Lição: Decisões antigas e estruturais têm efeito cascata. 'Uma implementação, por melhor que seja, não conserta uma decisão estrutural equivocada.'*

• **Amazon (2002):** Diretriz da liderança obrigando todos os times a exporem dados estritamente via interfaces de serviço bem definidas sem atalhos. Deu origem à AWS. *Lição: Decisões disciplinadas sustentam o crescimento por décadas.*

• **Trade-offs e Atributos:** Desempenho (Tráfego Aéreo), Segurança (Bancos), Escalabilidade (Apps de Hipercrescimento), Modificabilidade (SaaS contínuo).

1.6 Habilidades e Papel do Arquiteto

O arquiteto atua no sistema como um todo. Suas 3 habilidades centrais são: **Reunião** (entender quem paga pelo sistema), **Negociação** (conciliar times com interesses opostos) e **Tradução** (transformar requisitos vagos em metas verificáveis e quantitativas). Trabalha com apoio de DBAs, Analistas de Segurança e Engenheiros de Teste.

MÓDULO 2 • TIPOS DE SISTEMA E NÍVEIS DE ABSTRAÇÃO (MODELO C4)

2.1 Os 7 Domínios de Aplicação de Software

Domínio	Foco Principal e Preocupações Típicas	Exemplos
1. Software de Sistema	Atende outros programas; falha derruba as camadas acima.	SO, Drivers, Compilador javac
2. Software de Aplicação	Resolve necessidades de negócio; peso em regras e longo prazo.	Folha de pagamento, Acadêmico
3. Engenharia / Científico	Cálculo pesado; prioriza desempenho e corretude matemática.	Previsão do tempo, Estresse veicular
4. Software Embarcado	Reside em produto físico; pouco recurso e tempo de resposta rígido.	Freio ABS, Piloto automático
5. Linha de Produtos	Componentes reutilizáveis com flexibilidade de parametrização.	Cardápio genérico multi-cliente
6. Web e Móvel	Navegadores, apps e nuvem; atende muitos usuários simultâneos.	E-commerce, Apps bancários
7. Inteligência Artificial	Heurísticas para problemas complexos; fluxo de dados e inferência.	Reconhecimento facial, Antifraude

2.2 Níveis de Abstração e o Modelo C4 (Simon Brown)

Criado por Simon Brown, o modelo C4 trata a arquitetura como um **mapa com múltiplos níveis de zoom progressivo**. A cada zoom, surgem mais detalhes técnicos e oculta-se o panorama amplo:

• **Nível 1 - Contexto:** Mostra o sistema no centro (como caixa-preta) e seus relacionamentos com usuários (personas) e sistemas externos integrados. Não revela front/back nem bancos.

• **Nível 2 - Contêiner:** Abre o sistema para mostrar as unidades executáveis ou de armazenamento (SPA web, App móvel, API backend, Banco de dados, Fila de mensagens).

• **Nível 3 - Componente:** Abre um contêiner específico e detalha seus blocos funcionais internos coesos (controladores, serviços de negócio, repositórios).

• **Nível 4 - Código:** Detalha a implementação (diagramas de classes). Nem sempre exigido, pois o próprio código-fonte serve como especificação.

• **Onde a arquitetura para?** Critério do custo de reversão: *Quanto mais caro de reverter uma decisão, mais alto na escada de abstração e mais arquitetural ela é.*

MÓDULO 3 • ESTILOS ARQUITETURAIS FUNDAMENTAIS

3.1 Estilo Arquitetural (Macro) vs Padrão de Projeto GoF (Micro)

Um **Estilo Arquitetural** opera no nível macro de todo o sistema (quem conversa com quem e regras gerais). Um **Padrão de Projeto (GoF)** opera no nível micro, resolvendo problemas pontuais de estrutura de classes dentro de uma peça. Um sistema inteiro segue um estilo macro, mas pode conter dezenas de padrões GoF internos.

3.2 Quadro Comparativo dos 6 Estilos

Estilo	Ideia Central & Analogia	Ponto Forte	Riscos / Desafios	Exemplo Real
Monolítico	Toda a lógica em uma única unidade implantável. *(Canivete Suíço)*	Simplicidade de deploy e início.	Deploy arriscado no crescimento, difícil escalar partes isoladas.	Shopify (Monólito Modular em Rails)
Cliente-Servidor	Dois papéis: quem pede e quem responde. *(Restaurante)*	Modelo simples e amplamente difundido.	Servidor é gargalo e ponto único de falha.	DNS (stateless na internet)
Distribuído	Várias máquinas coordenadas agindo juntas. *(Correios)*	Escalabilidade massiva e resiliência.	Latência de rede, falhas parciais, consistência difícil.	Discord (Cluster Elixir/Erlang)
P2P (Ponto a Ponto)	Sem servidor central; nós são clientes e servidores. *(Troca de livros)*	Sem dependência central; custo zero de servidor.	QoS variável, moderação e segurança difíceis.	BitTorrent (Peers, Seeds, Tracker)
Pub-Sub	Publicação em tópicos intermediários sem conhecer assinantes. *(Assinatura de revistas)*	Altíssimo desacoplamento espaço-temporal.	Rastreamento complexo do fluxo de eventos.	RabbitMQ no Cientista Sem Jaleco
Pipes e Filtros	Processamento em sequência de etapas independentes. *(Estação de água)*	Filtros reutilizáveis e intercambiáveis.	Atraso acumulado; formato mal definido quebra fluxo.	Unix CLI (grep | sort) e Compiladores

MÓDULO 4 • ARQUITETURA EM CAMADAS (LAYERS)

4.1 As Três Camadas Clássicas

• **Apresentação (UI / Frontend):** Lida com quem consome a aplicação e faz a tradução entre o mundo externo e o interno. Recebe dados e valida formato.

• **Domínio (Serviços / Regras de Negócio):** Onde vivem as regras do negócio. Decide o que é permitido fazer e em quais condições. Calcula valores e orquestra validações.

• **Dados (Persistência / Repositórios):** Conversa com o banco de dados. Apenas salva e busca, sem tomar decisões de negócio.

4.2 O que Ganhamos com Camadas?

• **Testabilidade & Manutenibilidade:** Isola o domínio para testes unitários com mocks; problemas de negócio têm lugar certo para correção.

• **Nem tudo precisa mudar junto:** Mudanças de layout afetam Apresentação; novas regras afetam Domínio; troca de banco afeta Dados.

• **Baixo Acoplamento:** Comunicação via interfaces padronizadas *(Analogia da tomada elétrica)*.

• **Alta Coesão:** Cada camada foca no seu assunto. *Teste da Frase:* Tente descrever a unidade sem usar a palavra 'E'. Se precisar de vários 'e', faz coisas demais.

• **Cuidados:** Quebrar em mais partes não reduz acoplamento se amarradas; baixo acoplamento não é isolamento total (comunicação controlada é necessária).

4.3 Camada Lógica vs Física e Camadas Abertas vs Fechadas

- **Lógica vs Física:** Lógica é organização no código; Física (Tier) é o processo/máquina onde roda (1 processo no Monólito, 2 no Cliente-Servidor, 3+ no Distribuído).
- **Camada Fechada:** Chamadas passam obrigatoriamente pela camada imediatamente abaixo (mais seguro e padronizado).
- **Camada Aberta:** Permite pular intermediários para ganho de desempenho em consultas diretas (trade-off consciente).

4.4 Anti-padrões Comuns em Camadas

Anti-padrão	Como se Manifesta no Código
Apresentação Inteligente (Smart UI)	A tela assume regras de negócio, validações pesadas e consultas diretas ao banco de dados.
Domínio Anêmico	A camada de negócio vira um mero 'bypass' vazio de regras; a lógica fica dispersa na UI ou no banco.
Persistência Sabendo Demais	A camada de dados aplica regras de negócio na hora de montar queries ou salvar registros.
Vazamento de Camada (Layer Leaking)	Detalhes técnicos específicos (nomes de colunas SQL, exceções de banco) vazam para a Apresentação.

MÓDULO 5 • PADRÃO MVC E ORGANIZAÇÃO DE PASTAS

5.1 Estilo vs Padrão Arquitetural

Estilo: Caracteriza a organização macro sistêmica (quem conversa com quem de fora para dentro).
Padrão Arquitetural: Define papéis específicos (Model, View, Controller), atribui responsabilidades e regras de comunicação interna num contexto específico. *Estilo e Padrão são eixos independentes: Monólitos e Servidores Cliente-Servidor podem usar MVC por dentro.*

5.2 Papéis e História do MVC

- **Model:** Guarda os dados e a lógica que opera sobre eles (preço, estoque, descontos).
- **View:** Exibe os dados formatados para o usuário.
- **Controller:** Traduz o clique/ação do usuário em operações sobre o Model.
- **História:** Criado por **Trygve Reenskaug** (1978-1979 no Xerox PARC, projeto Dynabook de Alan Kay). Nome provisório: *Thing-Model-View-Editor* (maio/1979) -> *Model-View-Controller* (dez/1979).
- **Evolução:** Smalltalk-80 (1983) -> JSP Model 2 (1999, Web stateless request/response) -> Spring MVC (2004) -> Ruby on Rails (2004, convenção sobre configuração) -> React (2013, View dinâmica no browser SPA).

5.3 MVC Qualificado nas Três Camadas

No MVC clássico, o Model misturava regras de negócio e dados. No **MVC Qualificado moderno**:

- **Apresentação:** View + Controller.
- **Domínio:** Service (contém a lógica de negócio).
- **Dados:** Repository (persistência) + Entity (representação da tabela).
- **Tráfego: DTO** (Data Transfer Object) trafega entre Controller e View; **Entity** trafega entre Service e Repository.

5.4 Estruturas de Pastas: Package by Layer vs Package by Feature

Critério	Package by Layer	Package by Feature
Organização	Por papel técnico: controller/, service/, repository/, view/	Por funcionalidade de negócio: user/ (controller, service, repo, view)
Navegação	Fácil achar arquivos do mesmo tipo técnico.	Fácil enxergar tudo de uma mesma funcionalidade.

Extração p/ Microsserviço	Fica espalhado em várias pastas; alto atrito para migrar.	Já nasce coeso e isolado; migração rápida.
Indicação Ideal	Projetos menores, MVPs e equipes técnicas pequenas.	Projetos maiores, múltiplos times e features independentes.

Dica de Ouro para a Prova: *Lembre-se de que Arquitetura é sobre* ***estruturas e trade-offs****. Nenhuma tecnologia resolve arquitetura sozinha; o que define o sucesso a longo prazo é a disciplina de separação de responsabilidades, o baixo acoplamento e a alta coesão.*